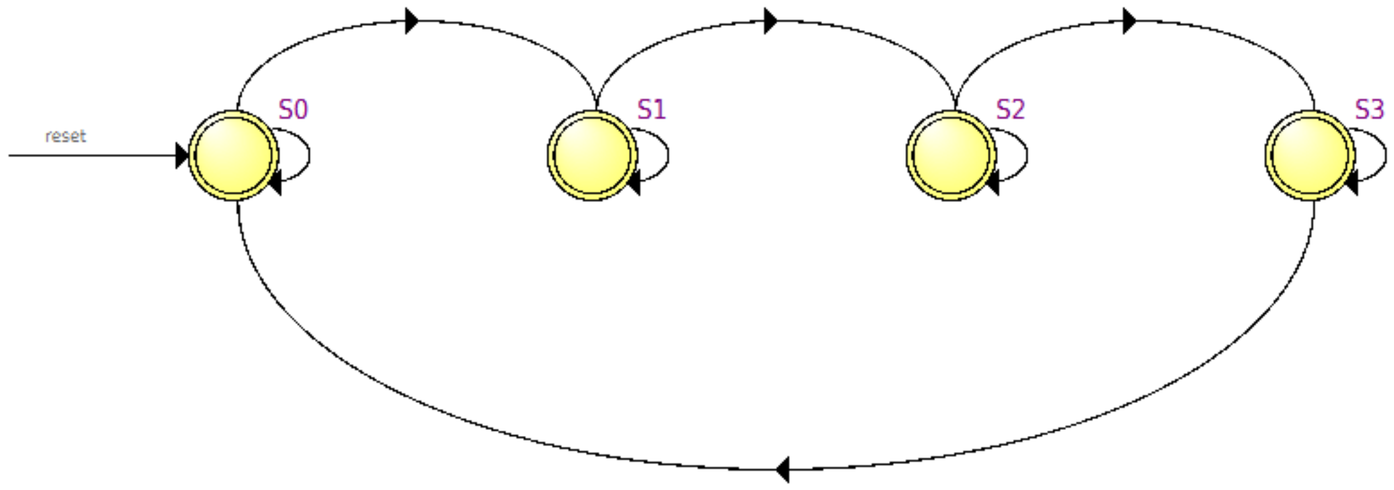
Mini-Project:
Traffic Light Controller in Verilog

By :
Abishek S
xia2020.abisheks@gmailcom

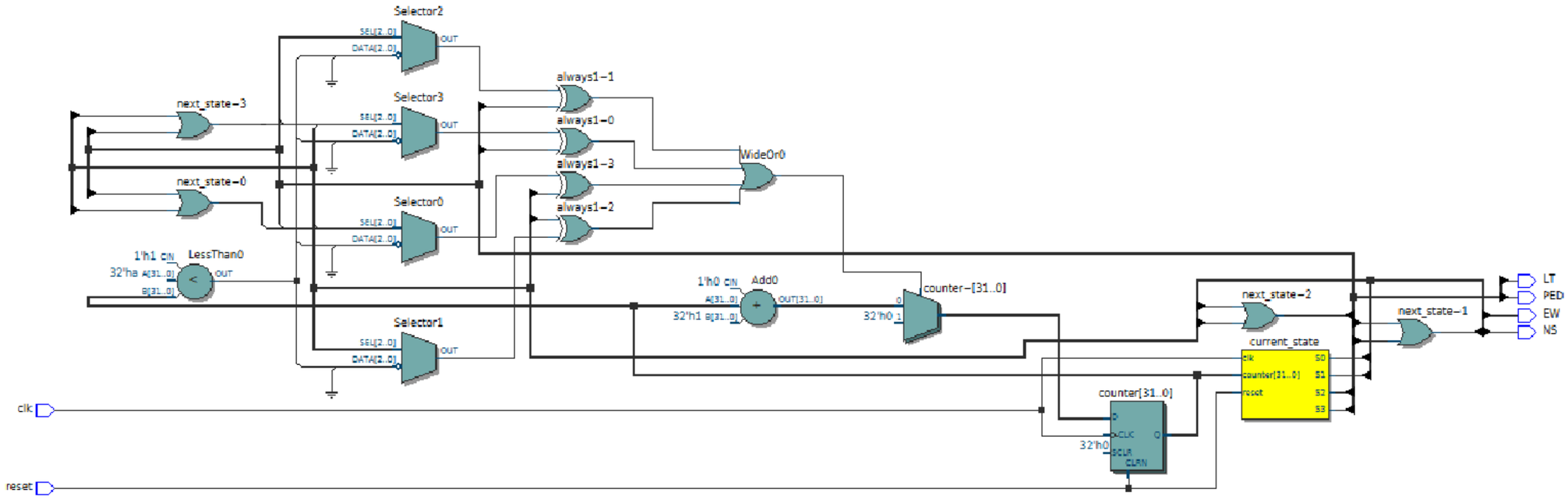
Aim : To design the Verilog HDL for the traffic light controller .

Tools : Quartus Prime lite 2025.1

Fsm State diagram :



RTL Viewer:


Selector2
Selector3
Selector0
Selector1
next_state~3
next_state~0
always1~1
always1~0
always1~3
always1~2
WideOr0
1'h1
LessThan0
32'ha
1'h0
Add0
32'h1
counter~[31..0]
32'h0
next_state~2
next_state~1
current_state
counter[31..0]
clk
reset
LT
PED
EW
NS

RTL Code :

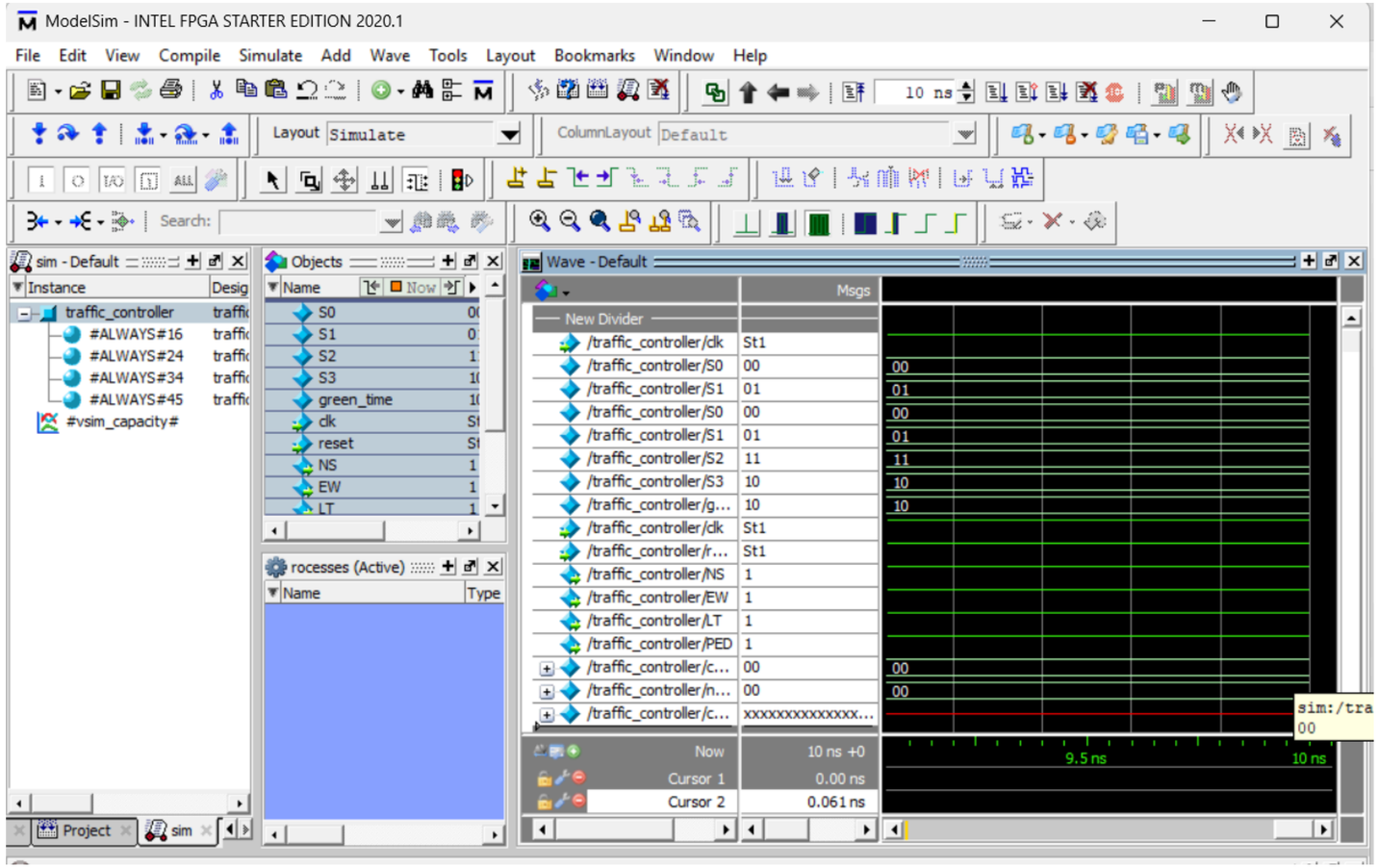
```
module traffic_controller(
      input clk,
      input reset,
      output reg NS,EW,LT,PED);

parameter S0=2'b00;
parameter S1=2'b01;
parameter S2=2'b11;
parameter S3=2'b10; //gray coding

reg [1:0] current_state, next_state;
reg [31:0] counter;

parameter green_time =10; //delay

always @(posedge clk or posedge reset)
       begin
           if(reset)
              current_state<=S0;
           else
              current_state<=next_state;
       end

always @(posedge clk or posedge reset)
        begin
            if(reset)
                counter <=0;
            else if (current_state!=next_state)
                counter <=0;
            else
                counter<=counter+1'b1;
         end
```

```
always @(posedge clk or posedge reset)
begin
    case(current_state)
       S0: next_state =(counter>=green_time) ? S1:S0;
       S1: next_state =(counter>=green_time) ? S2:S1;
       S2: next_state =(counter>=green_time) ? S3:S2;
       S3: next_state =(counter>=green_time) ? S0:S3;
       default: next_state = S0;
    endcase
end

always @(*) begin
    NS  = 1'b0;
    EW  = 1'b0;
    LT  = 1'b0;
    PED = 1'b0;

    case(current_state)
        S0: NS  = 1'b1;
        S1: EW  = 1'b1;
        S2: LT  = 1'b1;
        S3: PED = 1'b1;
    endcase
end

endmodule
```

Testbench :

D:/altera_lite/projects/traffic_controller.v | D:/altera_lite/projects/traffic_tb.v

```
`timescale 1ns/1ps

module traffic_tb;

reg clk;
reg reset;

wire NS, EW, LT, PED;

// Instantiate your design
traffic_controller uut (
    .clk(clk),
    .reset(reset),
    .NS(NS),
    .EW(EW),
    .LT(LT),
    .PED(PED)
);

// Clock generation (10ns period)
always #5 clk = ~clk;

initial begin
    // Initialize
    clk = 0;
    reset = 1;

    // Hold reset for some time
    #20;
    reset = 0;

    // Run simulation long enough to see state changes
    #500;

    $stop;
end

endmodule
```



Result : Since the traffic controller is designed and verified using quartus and modelsim.